

Real-Time Piano to Score Transcription

David Hadi

June 2026

Abstract

1 Introduction

Accurate piano transcription is a difficult challenge within the field of Digital Signal Processing (DSP), but the last decade has seen tremendous improvement due to the progress of machine learning models and curated datasets. Offline audio to score transcription is now a mature field—Zeng et al.[13] is the current state-of-the-art audio to score model—but such transcribers assume full context of the input audio. Real-time transcription has been approached in Kwon [7] and Wei [12], both reaching 0.97 F1 within 400 ms with their streaming models. Most recently, Hu fixed latency under 30 ms with a casual-AMT model, noting that musical applications require such time frames to be imperceptibly different from real-time [5]. However, all have gone with the approach of audio to MIDI (Musical Instrument Digital Interface). Our implementation connects the apparent gap, allowing for automatic transcription from audio to sheet music, as the audio is streaming. This was achieved through a combination of 1. a neural model that takes in log-mel spectrograms and produces note events, and 2. decoding and post-processing that convert the note events to notation.

2 Methodology

2.1 Overview

The challenge with using machine learning models to attack real-time musical transcription is the non-casual nature of most model architectures, and its conflict with the accuracy of the model predictions. While achieving near perfect F1 score, offline models require complete temporal context, requiring fully pre-recorded audio, and thus acting very slowly in its real-world implementations. On the other hand, streaming models by definition achieve very low latencies, but are infeasible due to their low accuracy. Our model takes a middle ground, similar to that of Wei [12]. We extract chunks of audio from the live recording

and process them with our low-latency model. Each chunk is spaced apart by 50 ms, but is 1.8 s in length; however, predicted onsets are only considered correct after two consecutive chunks confirm it, so we achieve 100 ms latency for this step. Since our implementation is unique in its scope (finalizing at sheet music instead of MIDI), the accuracy of that final product is up to human interpretation. However, when the model is run on MAESTRO test sets using our fast-chunking method, it achieves a `mir_eval` F1 score of 0.959, which is in the ballpark of SOTA models.

2.2 Input Features

Our model accepts a 229-bin log-mel spectrogram of the audio as the input feature with dimensions (T, 229), where T is measured in frames. We tested a range of input features, including CQTs, multiple mel-spectrograms, and others which are detailed in the Appendix, but found that the standard set by Hawthorne produced the best results [3]. To increase the speed of the spectrogram extraction, we resample the audio to 16 kHz before applying the STFT. We chose a Hann window of size 2048 with a hop length of 256; at a 16 kHz sample rate, this makes each window 128 ms long and each frame 16 ms long. Thus, for 10-second training clips the input feature dimensions are about (626, 229), and the 1.8-second chunks are (113, 229). However, an auxiliary dimension is allocated for batch number to allow for parallel processing (i.e., (1, 626, 229)). Since this dimension is not relevant to the processing of note events, it will often be omitted.

2.3 Training Objectives

Onset classification is challenging in that the vast majority of frames do not contain notes; thus, a naive binary classification model will learn to predict silence (this imbalance is why the standard evaluation of accuracy comes not in the form of percentages, but F1 scores). Thus, in choosing the structure of our model, we wanted to amplify the true onset predictions. So, we chose to label targets in a tent-shape around the ground truth onset/offset, linearly interpolating from 1 to 0 within a 25 ms radius around the gt [6]. Additionally, we tuned several hyperparameters that act as weights for the loss, which will be described as they arise.

Onset and offset losses are defined as such:

$$L_{head} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} SampleWeight(p, y_i) \cdot BCE(\sigma(x_i), y_i)$$

with

$$SampleWeight(p, y_i) = 1 + (p - 1)y_i$$

and

$$\mathcal{M} = \{i \in \mathbf{y} : \text{already predicted} = True\}$$

where $\mathbf{x}$ and $\mathbf{y}$ are the logits and target tensors, respectively, and p is defined as the *pos weight*, which amplifies positive event frames with *SampleWeight*. The weight is linearly interpolated due to the tent-shaped targets and prevents loss from being dominated by the non-onset/offset frames. Additionally, when the model is fine-tuned on the implementation in the live pathway, we mask already-predicted frames to prevent them from contributing to the loss, concentrating training on the newest predictions. However, this only pertains to that scenario; the original training run used the 10 s audio clips, and thus don't use the mask ($\mathcal{M} = \mathbf{y}$ for most training runs).

The frame head is almost equivalent, but with an additional focal weight:

$$L_{frame} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} Focal(p_t, \gamma) \cdot S.W.(p, y_i) \cdot BCE(\sigma(x_i), y_i)$$

where

$$Focal(p_t, \gamma) = (1 - p_t)^\gamma$$

where p_t is the probability that the the model's prediction matches the ground truth: that is,

$$p_t = \begin{cases} \sigma(x_i) & \text{if } y_i > 0.5 \\ 1 - \sigma(x_i) & \text{if } y_i < 0.5 \end{cases}$$

and γ is the *focal weight*, a hyperparameter which increases the weight of hard frames ($p_t \approx 0$) on the frame loss with the intention to encourage greater confidence. We experimented with different values for γ and found that $\gamma = 1$ gave the highest accuracy; higher values made it difficult for the model to improve. The frame *SampleWeight* is determined with a piecewise function

$$S.W.(p, y_i) = \begin{cases} p & \text{if } y_i > 0.5 \\ 1 & \text{if } y_i < 0.5 \end{cases}$$

Note that the function behaves identically to the linearly interpolated version at binary values (0 or 1). Since those are the only values the frame targets can take on, the weight is functionally equivalent to the onset/offset ones. The sounding frame loss is calculated in the same manner, so we omit writing it out for brevity.

The velocity head predicts a velocity magnitude between 0 and 127 (normalized to $[0, 1]$ for consistency), and thus uses mean-squared error:

$$L_{velocity} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} MSE(x_i, y_i)$$

however, the mask includes the additional condition that the frame must be active; otherwise, the head would overload on silent frames in a similar fashion to the onset/offset challenge.

The pedal head is unique among the heads in that it derives directly from the global conformer block, as it exists independently from any of the mentioned ones and requires complete context. (An aside: when we say "pedal", we mean the sustain pedal, as we did not explore the prediction of damper or sostenuto pedals. Perhaps this can be a future endeavor.) The MAESTRO dataset provides extremely articulate information about the pedal, giving it a magnitude number in the range $[0, 127]$; however, we only need per-frame binary information to inform our sounding frame head (more on this in Model Architecture). We decided on a hybrid approach where the model predicts a value between 0 and 127, but all decisions based on the pedal head are thresholded at 64, rendering the head binary. We believe this is justified; firstly, the MAESTRO pedal data is bimodal at the two extremes, meaning that it is almost already a binary value; secondly, the only use of the pedal head is in conjunction with the frame head to construct the sounding frame head. Thus, using (for example) an MSE for pedal loss would be unnecessary and of ambiguous utility. So, we defined it in the following manner:

$$L_{pedal} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} S.W.(p, y_i) \cdot BCE(\sigma(x_i), y_i)$$

with *SampleWeight* acting similarly to the piecewise defined in the frame loss:

$$S.W.(p, y_i) = \begin{cases} p & \text{if } y_i > 64/127 \\ 1 & \text{if } y_i < 64/127 \end{cases}$$

where we divide the threshold by 127 (it is essentially 0.5) for normalization.

The total loss is calculated as a weighted sum of these individual head losses:

$$L_{tot} = \sum_{heads} w_{head} \cdot L_{head}$$

where the weights are proportional to the importance of each head on the model's training objective:

Table 1: Loss weights for the enhanced piano-transcription model

Output head	Training target	Loss weight
Refined onset	Note-on event	1.00
Auxiliary raw onset	Note-on event	0.25
Refined offset	Note-off event	1.00
Auxiliary raw offset	Note-off event	0.25
Frame	Key-active frame	0.80
Sounding frame	Acoustically sounding frame	0.30
Pedal	Sustain-pedal state	0.10
Velocity	MIDI velocity	0.30

2.4 Model Architecture

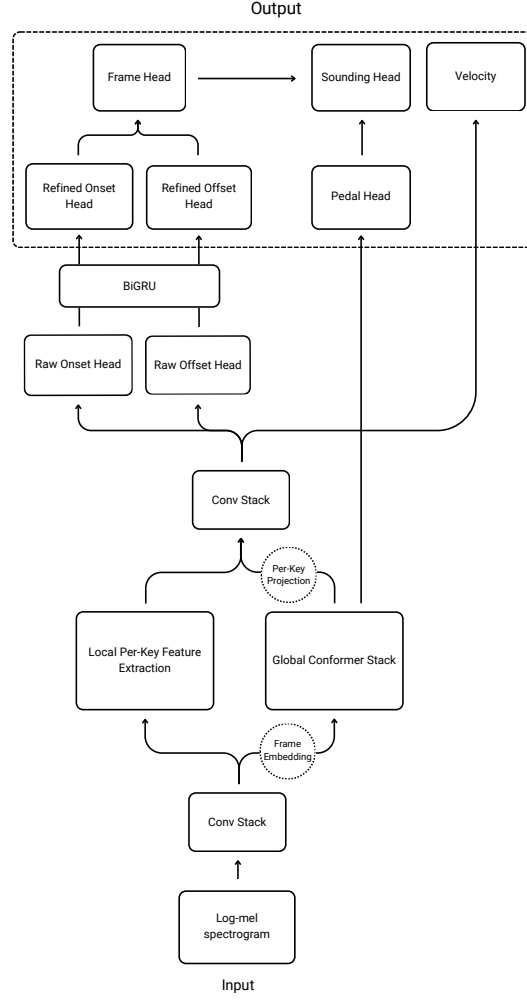


Figure 1: Model Diagram

The diagram in Figure 1 shows the model configuration. The input features are passed into a convolutional stack based on the VGG architecture (see Appendix). This stack acts on the mel-frequency axis, creating a feature map. Then, the map is passed into a local feature extraction for each of the 88 piano keys; at the same time, it is flattened along the frame dimension and passed through 10 conformer blocks for temporal context, allowing the model to take advantage of both per-key features and global context. The paths' output is recombined with dimensions (B, T, 88, 96), and a dilated conv stack is run along each of

the 88 keys on the time axis to provide further per-key temporal refinement. The features are then passed into three MLPs for each of the three basic heads: onset, offset, and velocity. After initial predictions, a BiGRU is run over the predictions from the onset and offset heads, weighted by velocity, for learning cross-key interactions: similar in effect to the global conformer stack, but after preliminary predictions have already been made. The combination of the onset and offset head forms the frame head.

The "sounding head" is composed of the frame and pedal head, and denotes a choice we made about note durations. We noticed that pedal frequently lengthens the duration of a note when compared to the value written in by a meticulous and thoughtful composer, but their usage in music that is not yet written down—improvisations, or ongoing compositions—is much harder to evaluate and is subjective at best. We envision a use case for this technology that does not involve a set score, and so decided to fold the pedal durations into the note duration. This greatly decreases the accuracy of offsets and note durations in the model evaluation; however, this is of minor importance. The onset head controls rhythm and the model as a whole predicts pitch, which contribute more objectively to the accuracy of the transcribed score, especially as compared by a human to the music they just played.

2.5 Decoding

The model returns a dictionary of tensors for onsets, offsets, sounding-frame activation, and velocity, each with dimensions $[T, 88]$, where T is the time-axis, measured in frames, and 88 represents the number of keys on a standard piano. Additionally, the pedal activations (dimensions $[T,]$) and frame activations (dimensions $[T, 88]$) are returned for posterity, but never used in decoding; the only contribution is within the model to inform the sounding-frame head, which is used in place of the frame activation to determine duration. The onset, offset, and sounding-frame logits are then converted into probabilities by applying a sigmoid function. For velocity, the sigmoid returns a normalized value that is then multiplied by 127 to get the velocity predictions. It was preference whether to return the probabilities directly from the model or calculate them afterwards, but we chose this order; since all utilized logits are converted in this manner, they will be considered as the inputs into the decoding algorithm rather than the raw model outputs. The decoder iterates over each pitch, producing a list of note events for each one and concatenating them to form the output.

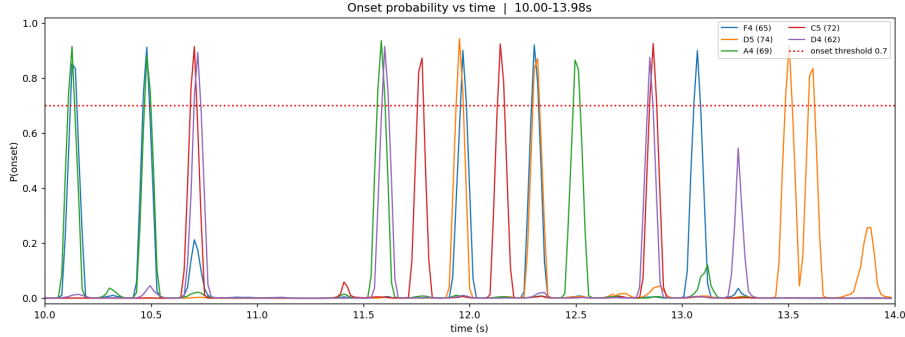


Figure 2: Onset probability distribution

Onset time is determined by iterating over frames and selecting local maxima of the onset probability distribution (Figure 2). We also set a threshold for the minimum probability necessary to even be considered (the dotted line); this heuristic was experimentally determined to achieve the maximum accuracy at 0.70. The value represents a tradeoff between recall and precision: too high, and recall will suffer as precision increases; too low, and the opposite is true. Hawthorne et al [3] chose a threshold of 0.50 as a "natural boundary", but we found that our higher value balances the tradeoff better than the obvious binary choice. We experimented with a secondary pass of onset frames that rescued lower probability onsets that took prior context and frame and velocity evidence into account; however, this was found to decrease precision without meaningfully increasing recall. This imbalance led us to conclude that recall is mainly controlled by the model itself, and recall-targeted decoding-side algorithms produced little improvement because the missing notes were never predicted by the model.

Offset time is determined through two algorithms; we choose whichever produces an offset earlier in time. The first is equivalent to the onset time algorithm, utilizing the offset probability distribution and an experimentally determined offset threshold. The second compares frame probabilities to a frame threshold, and takes the first frame that does not meet that threshold as the offset candidate. We also set a minimum duration of 32 ms (2 frames), with the assumption that it is mechanically impossible to play two distinct notes that quickly.

We attempted velocity-weighted calibration (i.e., using velocity values as confidence measures) as a secondary filter for both onsets and offsets, but found that it was unhelpful; in experiments, it did not meaningfully change F1, corroborating with findings from Hu [5].

Our implementation depends on fast chunking to achieve streaming-like outputs due to the discrete nature of our input features. To take advantage of this, we use a persistence-based filter over the preliminary onsets, which requires two

separate chunks to corroborate its existence. We reasoned that false positives tend to be specific to the context that surrounds them; therefore, viewing a predicted note event from two different context windows and confirming its existence over both decreases the chance that the prediction was ephemeral. Empirically, this method led to an increase in precision: from 0.59 with one observation (no persistence) to 0.97 with two. However, the strategy inherently costs two decode runs per note; in general, this leads to a latency that grows in $O(\text{number of observations})$. Additionally, the consensus of three or more windows produces rapidly diminishing returns at a heavy cost of latency (see the figure below).

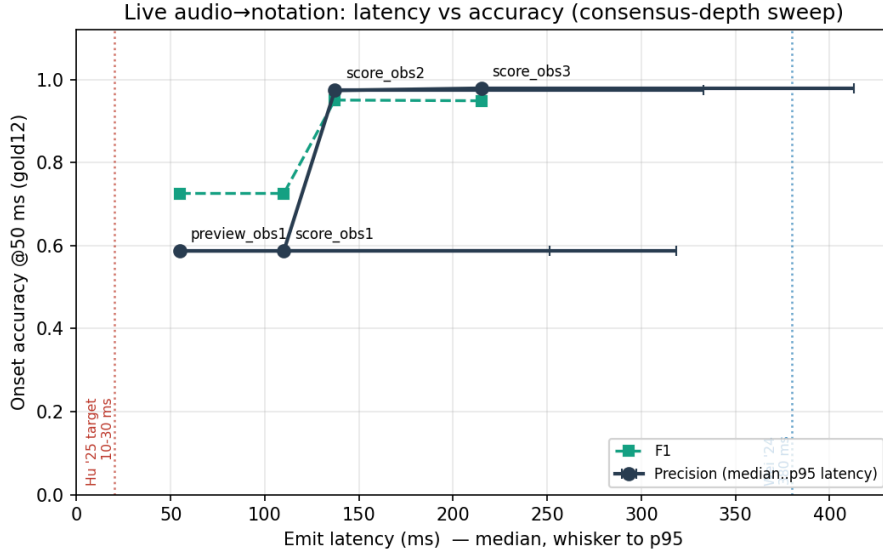


Figure 3: How observation count affects accuracy and latency

To help with the latency challenge, we produced another filter that only required one observation per note. It is well established in chunked inference that the edges of the chunk are the most inaccurate, due to the lack of context on either side [1][8][10]. As a result, a filter that only considers notes in the interior of the chunk can be assured that an ample amount of past and future context had been considered; our specific one is one-sided, as past context is generally present. However, we found that such a filter decreases latency at the cost of precision; while the persistence filter had 0.97 precision at a median latency of 137 ms (more on how this is calculated later), the interior filter had 0.94 precision at a median latency of 100 ms. We leave it up to debate which of accuracy and speed is more important for streaming-based AMT, but we ultimately chose the persistence filter after real-world testing.

2.6 MIDI to Score

The difficulty in converting note events to formatted sheet music lies in the rigidity that notes on a score have to conform to. Chords must be clustered and separated from other onsets; notes must be quarter, eighth, sixteenth, etc.; the period that should be considered a "quarter note" must be decided on through tempo; and notes must be separated into treble and bass voices. We address the complications described above—that is, chord notation, note value quantization, and voicing. These are only a small subset of a much larger category of problems with attempting AST, not least of which include broader varieties of each of the challenges we try to solve. However, given that we are working without full audio context and minimal look-ahead, we believe our implementation is pragmatic.

After decoding, onsets that are within a certain threshold of each other are grouped into chords. We elected for a tempo-independent cutoff of 20 ms, determined from the first note in the hypothesized chord; this allows us to capture groupings that may span slightly more than 1 frame, while still lying well below even the quickest monophonic striking capabilities. The chords then adopt the average onset time and the maximum duration of its constituents.

We want to mention tempo detection briefly due to its apparent necessity in note quantization. Tempo following is achievable in real-time with probabilistic state-space models [2][4]; however, it is impractical in our implementation. As live tempo changes, the quantization changes, which will lead to one of two problems: either we would have to reformat the entire existing score every time, or else suffer rhythm failure as parts of the score are quantized under different tempos. As such, we set a default tempo of 120 bpm, and we could add an additional, separate refinement option that predicts tempo from the full context and requantizes the entire score if a user so wished.

The note events are quantized to a 1/24 beat grid, allowing for precision up to 32nd triplets. This is then split into treble and bass clef at middle C (midi note 60). We recognized that this naive approach to voicing is lacking in some respects, but whether a note should be in treble or bass clef (and other voicings) is completely up to preference and musical intention. We only split in this way to produce a score that is relatively neutral, and, in practice, would rely on human editing to create intentional musical structure.

3 Results

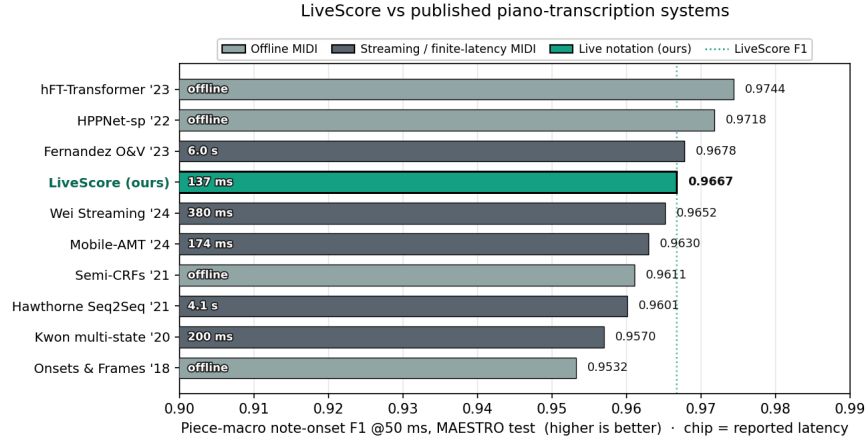
3.1 Experimental Setup

We utilized the MAESTRO dataset to curate a 48-clip subset, which our model is tested on. The *mir_eval* library is then used to evaluate our model in a standardized manner, allowing for accurate and fair comparison with other models in the field [11]. In particular, the standard tolerance for the maximum proximity to a GT note value to be considered accurate is ± 50 ms.

This evaluation covers the audio to MIDI portion of our methodology, but we also write to a score. Thus, we utilize the MV2H metric to measure score accuracy, which is an average of five separate metrics: multi-pitch, voice, value, harmony, and meter [9]. However, we only test on multi-pitch, value, and voice, for the reasons described in the last section.

Both of the evaluations are run on results from our live transcription pathway. We also include latency results in the following based on different ablations we performed, but for the two accuracy metrics we keep the latency constant by using the main pathway for everything.

3.2 Main Results



LiveScore is evaluated on all 177 full-length pieces in the official MAESTRO v3 test split; published values retain their papers' stated MAESTRO protocols. The chip is reported algorithmic latency. Hu Causal-AMT '25 (strict 10-30 ms tol) and Zeng A2S '24 (MV2H) use non-comparable metrics and are omitted here — see the latency-accuracy landscape and MV2H figures.

Figure 4: Mir_Eval Comparison

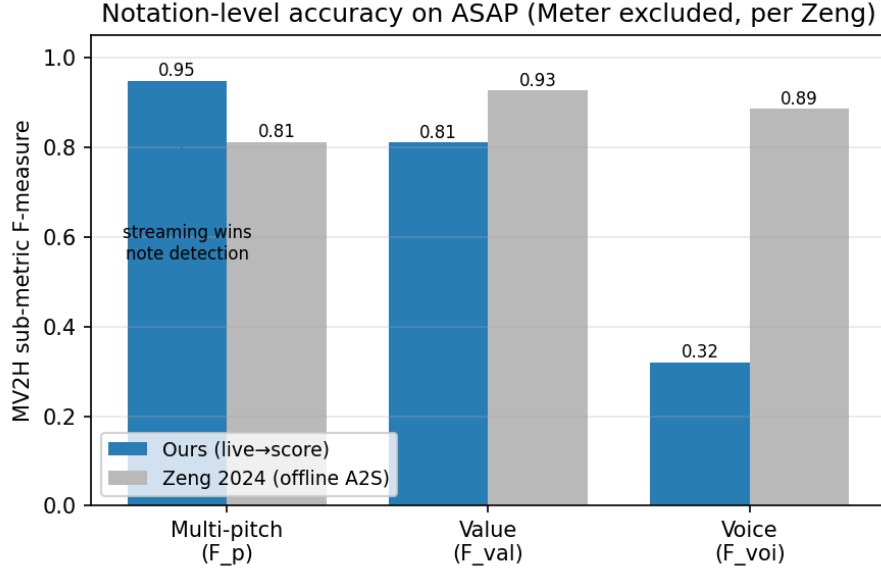


Figure 5: MV2H Comparison

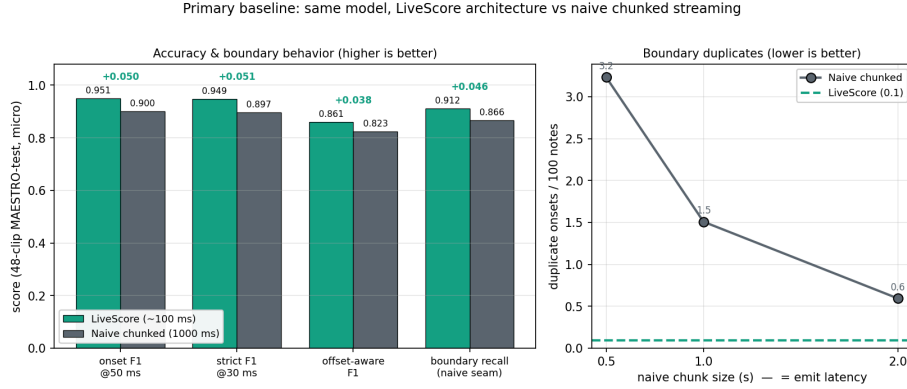


Figure 6: Self-Comparison (Our implementation vs. Naive chunked inputs)

3.3 Analysis

We are most interested in the effect of capture techniques used to improve accuracy and latency. By comparing our current architecture to a chunked baseline that also uses our model and decoding process, we are able to identify its role in the overall performance of the entire system. We found that the combination of the sliding capture window and double-observation persistence

check led to better performance overall, but especially in precision metrics. With the basic chunked system, boundary duplicates appear due to the false detection of onsets for notes split at the borders of chunks. Additionally, we find that there is a tradeoff between this precision loss and latency. Our implementation made boundary duplicates almost negligible in effect, at 1 duplicate per 1000 notes compared to as high as 30 duplicates per 1000, and with better latency.

To find the individual contributions of the sliding window and the persistence check, we ran a small ablation study (small because there are only two components). Since the persistence check is dependent on the existence of overlapping chunks, we ran three tests: one of the naive chunked baseline, with the sliding window included, and then finally with the persistence included. We found that the sliding window alone decreased precision by 0.245 and increased duplicate onsets by 10 notes per 1000; this was largely expected, as the many overlapping chunks with no correction would lead to a larger number of mistakenly detected onsets. Additionally, it increased recall by 0.073 at the boundaries. However, when paired with the persistence check, the precision cost of the sliding window is completely offset, leading to an overall precision that is 0.332 higher than the sliding window alone and 0.088 higher than the chunked baseline, and with 96% less duplicate onsets than the sliding window alone. It also kept much of the recall improvement, at +0.047 versus +0.073 compared to the baseline.

On the MV2H benchmark, our live system outperforms Zeng’s offline model [13] on multi-pitch by 0.14 points, but does worse on value (-0.12) and significantly worse on voice metrics (-0.57). However, this is consistent with our design. The duration of each note is determined by the logits from the sounding frame head, which was explained in more detail in the Methodology section. The voicing is split into treble and bass clef to provide maximum flexibility for users who want to edit the transcription afterwards. In both cases, the efficacy of the system is geared toward what the pianist actually played, rather than the score.

4 Conclusion

We present a live A2S system comprised of a capture technique, a multi-head model, and a decoding sequence that leads to an improvement in latency and accuracy over similar SOTA systems. The system is particularly disposed towards pianists that want to create original music (i.e., composers and improvisers), and we are currently working to integrate the system into a functioning app that may help speed up piano transcription in the musical industry. In future work, we may continue to lower the per-note latency of audio transcriptions, with the goal of reaching sub-30 ms levels at a respectable accuracy as described by Hu [5] in his latency-based paper.

There were many negative results not covered in the main results section of this paper, mainly due to their tangential nature as well as the sheer quantity of them. To provide an analysis of each would be tiring and irrelevant, but all

tables of results gathered from each in their respective scopes are detailed in the Appendix. We highly recommend viewing them in their entirety if one wishes to embark on a similar research topic; in retrospect they would have been very helpful in weighing the many options to attack the transcription challenge.

5 Acknowledgments

The use of Claude Code and similar generative AI models were relied upon during research. Although the entire paper was written by us, and the ideating was done by us, we could not have implemented our concepts into code without AI, and for that we are grateful.

We would also like to thank (insert people) for helping in (insert ways).

6 Appendix

Table 2: Early representation and rhythm-quantization ablations on MAE-STRO. Higher is better. These experiments motivated the single-mel frontend and IOI-based duration quantization used in later experiments.

Method	Reference	Ablation	Outcome
Input representation	229-bin mel: $F_{on} = 0.73$	1098-D multi-resolution stack: 0.72	−0.01 onset F1
Rhythm post-processing	No coherence smoothing: 0.522	Coherence smoothing: 0.440	−8.2 points IOI accuracy
Duration source	IOI quantizer: 52.2%	Neural note-value head: 29.6%	−22.6 points accuracy

Table 3: Rejected recall-oriented decoding additions on the 48-clip live replay manifest. Small recall gains were outweighed by false positives and degraded score structure.

Method	Recall	Precision	Note F1	Cluster F1	P95 (ms)
Frame rising-edge recovery (off)	0.9237	0.7656	0.8318	0.6072	–
Frame rising-edge recovery (on)	0.9268	0.7409	0.8171	0.5967	–
Soft-polyphony rescue (off)	0.9434	0.9542	0.9488	0.7418	24.6
Soft-polyphony rescue (on)	0.9511	0.9251	0.9379	0.7137	32.7

Frame recovery reduced cluster F1 with a paired randomization test of $p = 0.0003$. Soft-polyphony rescue added recall but doubled the duplicate rate (0.227 to 0.480 per 100 notes).

Table 4: Negative ablations of live display-state policies on the production continuous-stream surface (Gold-12). The legacy count-based persistence gate was retained.

Policy	Precision	Recall	Note F1	Cluster F1	Duplicates/100
Legacy count gate	0.9534	0.9243	0.9386	0.7246	–
SPRT, observation-heavy	0.9091	0.9296	0.9192	0.6686	–
SPRT, recall-oriented	0.8704	0.8566	0.8635	0.5590	–
SPRT, precision-oriented	0.9606	0.6149	0.7498	0.5170	–
Baseline for onset/harmonic tests	0.9670	0.8566	0.9085	0.6881	0.45
15 ms onset snapping	0.9670	0.8566	0.9085	0.6830	0.30
Harmonic-prior promotion	0.9574	0.8645	0.9086	0.6819	0.44

The SPRT and harmonic-prior experiments reveal the same discriminator failure: soft true notes and persistent decode noise overlap in confidence and harmonic context.

Table 5: Manifest-scale negative results for display-surface chord repair. All values are control-arm display metrics on the 48-clip replay benchmark.

Method	Baseline cluster F1	Treatment cluster F1	Δ	Decision
Ambiguous-candidate guard	0.6072	0.6068	-0.0003	Rejected ($p = 0.8477$)
Duplicate-pair resolver	0.6072	0.6074	+0.0002	Not promoted ($p = 0.9365$)
9-pass duplicate-pair retraining	0.6072	0.6074	+0.0002	Not promoted
Sustained-note re-emission filter	0.6072	0.6073	+0.0001	Rejected
Cluster-aligned pitch-set editor	0.6072	0.5874	-0.0198	Rejected ($p < 0.0001$)
Neural-probability pitch feature	0.6115	0.6088	-0.0028	Rejected

The neural-probability feature raised training pitch AUC from 0.6252 to 0.9173, but reduced manifest display-note F1 from 0.8601 to 0.8564 ($p = 0.0081$), illustrating a training-to-deployment mismatch.

Table 6: Latency-policy frontier on the continuous streaming path. Reducing the number of corroborating observations below two causes a precision cliff; reducing the inference interval below 50 ms makes observations too correlated to act as independent confirmation.

Configuration	Approx. latency (ms)	Precision	Recall	$F_1@50$
3 observations, 70 ms interval	210	0.984	0.918	0.950
2 observations, 50 ms interval	100	0.982	0.922	0.951
1 observation, 80 ms interval	80	0.575	0.944	0.715
2 observations, 40 ms interval	80	0.863	0.932	0.897

Table 7: Voice-lane ablation on Gold-12 score-edit accuracy. More nominal voices did not improve the voiceless reference evaluation; per-hand lanes were therefore used as the practical score-duration policy.

Voice assignment	Score-edit accuracy	Interpretation
Pitch lanes (previous default)	28.62%	Fragmented melodic lines when they crossed pitch buckets.
Two streaming lanes	24.2%	Worse than the pitch-lane baseline.
Three streaming lanes	22.4%	Further regression.
One lane per hand	41.10%	Best match to the available voiceless reference, but not a claim of full polyphonic voice separation.

Table 8: Sensitivity of onset-cluster F1 to realistic onset jitter. Pairwise co-onset F1 is substantially more stable than single-linkage cluster F1 and was used as a diagnostic rather than further optimizing a jitter-sensitive cluster boundary.

Metric	No jitter	5 ms jitter	10 ms jitter	15 ms jitter
Single-linkage 50 ms cluster F1	1.000	0.928	0.859	0.766
Pairwise co-onset F1	1.000	0.979	0.943	0.904

References

- [1] Keyu An et al. “CUSIDE: Chunking, Simulating Future Context and Decoding for Streaming ASR”. In: *Proc. Interspeech 2022*. 2022, pp. 2103–2107.
- [2] Ali Taylan Cemgil et al. “On Tempo Tracking: Tempogram Representation and Kalman Filtering”. In: *Journal of New Music Research* 29.4 (2000), pp. 259–273. DOI: 10.1080/09298210008565462.
- [3] Curtis Hawthorne et al. “Onsets and Frames: Dual-Objective Piano Transcription”. In: *arXiv preprint arXiv:1710.11153* (2018).
- [4] Mojtaba Heydari, Frank Cwitkowitz, and Zhiyao Duan. “BeatNet: CRNN and Particle Filtering for Online Joint Beat, Downbeat and Meter Tracking”. In: *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*. Online, 2021, pp. 417–424. arXiv: 2108.03576 [cs.SD].
- [5] Patricia Hu et al. “Exploring System Adaptations for Minimum Latency Real-Time Piano Transcription”. In: *Proceedings of the 26th International Society for Music Information Retrieval Conference (ISMIR)*. Daejeon, South Korea, 2025.
- [6] Qiuqiang Kong et al. “High-Resolution Piano Transcription with Pedals by Regressing Onset and Offset Times”. In: *arXiv preprint arXiv:2010.01815* (2020). arXiv: 2010.01815 [cs.SD].

- [7] Taegyun Kwon, Dasaem Jeong, and Juhan Nam. *Towards Efficient and Real-Time Piano Transcription Using Neural Autoregressive Models*. 2024. arXiv: 2404.06818 [cs.SD].
- [8] Xilai Li et al. “Dynamic Chunk Convolution for Unified Streaming and Non-Streaming Conformer ASR”. In: *ICASSP 2023 – IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2023.
- [9] Andrew McLeod and Mark Steedman. “Evaluating Automatic Polyphonic Music Transcription”. In: *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*. Paris, France, 2018, pp. 42–49. URL: https://ismir2018.ismir.net/doc/pdfs/148_Paper.pdf.
- [10] Vahid Noroozi et al. “Stateful Conformer with Cache-based Inference for Streaming Automatic Speech Recognition”. In: *ICASSP 2024 – IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. arXiv:2312.17279. 2024.
- [11] Colin Raffel et al. “mir_eval: A Transparent Implementation of Common MIR Metrics”. In: *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*. Taipei, Taiwan, 2014, pp. 367–372.
- [12] Weixing Wei et al. *Streaming Piano Transcription Based on Consistent Onset and Offset Decoding with Sustain Pedal Detection*. 2025. arXiv: 2503.01362 [cs.SD].
- [13] Wei Zeng, Xian He, and Ye Wang. “End-to-End Real-World Polyphonic Piano Audio-to-Score Transcription with Hierarchical Decoding”. In: *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24)*. 2024.